SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-020: Student Self-Service Password Change

Student Management System

Field	Details
Change Request ID	CR-020
Change Request Title	Student Self-Service Password Change
Student Name	Y.Ugatharsini
Student ID	JF/ICT/24/33
Implementation Status	Completed
Date	13 October 2026
System Version	Student Management System v1.1
Priority	Medium
Difficulty	Medium
Change Type	Frontend and Backend Enhancement

1. Change Request Summary

1.1 Business Requirement

The Student Management System required a facility that allows students to change their account passwords independently through the Student Dashboard.

Previously, students depended on the existing authentication functionality to access their accounts. A dedicated self-service password change facility was required so students could update their passwords without requiring an administrator to modify their account credentials.

The new feature allows an authenticated student to enter their current password, provide a new password, and confirm the new password. The system validates the submitted information, verifies the current password against the stored password hash, and securely stores the new password hash.

The Student Dashboard was also enhanced to provide a Change Password button. The password change form remains hidden when the dashboard initially loads and appears only when the student clicks the button. The student can close the form using the Cancel button or the toggle button.

The implementation aims to improve account security, usability, and student independence while preserving existing authentication, student profile, and administrator functionality.

1.2 Existing Behaviour

Before implementation:

- Students could log in using their registered usernames and passwords.
- Password verification was handled by the existing authentication controller.
- Passwords were stored as hashes in the users table.
- The Student Dashboard displayed student profile information.
- The Student Dashboard did not provide a dedicated self-service password change form.
- Students could not change their passwords through the Student Dashboard.
- No dedicated interface was available for entering the current password, new password, and password confirmation.
- The system did not provide a dedicated password change workflow for authenticated students.

1.3 Requested Change

The requested change was to introduce a secure student password change facility.

The following functionality was required:

- Add a Change Password button to the Student Dashboard.
- Display the password change form only after the button is clicked.
- Allow the student to enter their current password.
- Allow the student to enter a new password.
- Provide a confirmation field for the new password.
- Validate the required fields before submitting the request.
- Enforce the existing password strength requirements.
- Verify the current password before updating the account.
- Ensure that the new password and confirmation match.
- Prevent the new password from being identical to the current password.
- Hash the new password before storing it in the database.
- Restrict the password change operation to the authenticated student's own account.
- Display appropriate success and error messages.
- Allow the student to cancel and close the form.
- Clear password fields after a successful password change.
- Preserve existing login, logout, profile retrieval, and administrator functionality.

2. Implementation

2.1 Implementation Summary

CR-020 was implemented by extending the existing Student Management System frontend and backend.

On the frontend, the Student Dashboard was updated with a Change Password button and a state variable that controls whether the password change form is visible. The form is displayed only after the student clicks the button. Clicking the toggle button again or selecting Cancel closes the form.

A new ChangeStudentPassword.jsx component was introduced to manage the password change form. It contains fields for the current password, new password, and confirmation password. Client-side validation provides immediate feedback before submitting the request to the backend.

On the backend, a new authenticated route was introduced:

PATCH /api/students/me/password

The route uses the existing authentication middleware and restricts access to users with the STUDENT role.

The password change controller obtains the authenticated student's user ID from the verified request token rather than trusting a user ID supplied by the frontend. It validates the submitted fields, retrieves the stored password hash, verifies the current password using bcryptjs, hashes the new password, and updates the student's password hash.

The existing user model was extended with functions to retrieve the password hash for an authenticated student and update the password hash.

No new database table or database column is required because the existing users.password_hash field is reused.

The implementation preserves the existing login process and student profile functionality.

2.2 Affected Files

File	Change	Purpose
backend/src/models/userModel.js	Modified	Adds functions to retrieve a student's stored password hash and update the password hash.
backend/src/controllers/studentController.js	Modified	Adds the changeMyPassword controller to validate the request and update the authenticated student's password securely.
backend/src/routes/studentRoutes.js	Modified	Adds the authenticated PATCH /me/password route restricted to the STUDENT role.
frontend/src/components/ChangeStudentPassword.jsx	Added	Provides the password change form, client-side validation, API request handling, and success/error messages.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Adds the Change Password toggle button and conditionally displays the form.
frontend/src/styles/studentDashboard.css	Modified	Adds styling for the Change Password button, form presentation, and responsive layout.

Files Retained Without Changes

The following existing files are retained without requiring changes for this implementation:

- backend/src/models/studentModel.js
- backend/src/controllers/authController.js
- backend/src/middleware/authMiddleware.js
- Existing database schema, including the users.password_hash column.

The existing validation utility is reused for password strength validation where applicable.

2.3 Database Modification

No database schema modification is required for CR-020.

The existing users table already contains the password_hash field used by the authentication system. The new feature updates this field with the securely hashed new password.

The following existing fields are relevant:

Field	Purpose
user_id	Identifies the student account.
username	Identifies the account during login.
password_hash	Stores the hashed password.
role	Identifies the account role, including STUDENT and ADMIN.

The implementation does not store plaintext passwords in the database and does not introduce a separate password storage table.

2.4 Technical Changes

2.4.1 Student Dashboard Modification

The StudentDashboardPage.jsx component was modified to include a state variable:

showChangePassword

This state controls the visibility of the password change form.

When the dashboard loads, the state is initially set to false, so the form is hidden.

When the student clicks the Change Password button, the state changes to true and the form is rendered.

When the student clicks the toggle button again, the state changes back to false and the form is hidden.

The button label changes according to the current state:

- Change Password when the form is hidden.
- Cancel Change Password when the form is visible.

The existing student profile, loading messages, error handling, and Logout functionality are retained.

2.4.2 Password Change Form

A new ChangeStudentPassword.jsx component was implemented.

The form contains three password fields:

1. Current Password
2. New Password
3. Confirm New Password

The component manages the password field values, loading state, validation messages, and API request result.

The form includes a Cancel button that closes the form without submitting the password change request.

After a successful password change, the component displays the success message and clears the password fields.

2.4.3 Client-Side Validation

Client-side validation was introduced to provide immediate feedback to students before submitting their requests.

The validation checks include:

- All required password fields must be completed.
- The new password must satisfy the configured password strength requirements.
- The new password and confirmation password must match.
- The new password must not be identical to the current password.

The configured password strength requirements are:

- At least eight characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character.

Client-side validation improves usability but does not replace backend validation. The backend independently checks the submitted information before changing the password.

2.4.4 Backend Route Modification

A new password change route was added to studentRoutes.js:

PATCH /me/password

The route is protected by the existing authentication middleware and student-role authorization.

The complete API path is:

PATCH /api/students/me/password

The route uses the authenticated student's identity from the verified token. It does not accept an arbitrary student ID from the request body.

This prevents the request from selecting another student's account by changing an ID in the submitted data.

2.4.5 Password Change Controller

The changeMyPassword controller was added to studentController.js.

The controller performs the following operations:

4. Receives the current password, new password, and confirmation password.
5. Checks that the required fields have been provided.
6. Validates the new password against the configured password strength rules.
7. Checks that the new password and confirmation match.
8. Rejects a new password that is identical to the current password.
9. Retrieves the authenticated student's user ID from req.user.userId.
10. Retrieves the stored password hash for the student's account.
11. Verifies the current password using bcrypt.compare().
12. Hashes the new password using bcrypt.hash().
13. Updates the stored password hash using the user model.
14. Returns an appropriate response indicating success or failure.

The controller does not return the stored password hash or expose password values in its response.

2.4.6 User Model Modification

The userModel.js file was extended with two functions.

`getPasswordHashByUserId(userId)`

Retrieves the password hash for the specified student account. The query restricts the operation to accounts with the STUDENT role.

`updatePasswordHash(userId, passwordHash)`

Updates the password hash for the specified student account.

The model uses parameterized SQL queries to separate supplied values from SQL statements.

The existing user lookup and last-login tracking functions are retained.

2.4.7 API Communication and Error Handling

The frontend submits the password change request through the existing API service.

The request contains the following fields:

- currentPassword
- newPassword
- confirmPassword

The backend validates the request before updating the database.

The application provides appropriate responses for validation failures, incorrect current passwords, and successful password changes.

The form also maintains a loading state during submission to prevent repeated submissions while the request is being processed.

2.4.8 User Interface and CSS Modification

The studentDashboard.css file was updated to style the Change Password button.

The styling includes:

- Blue button background.
- White button text.
- Rounded corners.
- Hover and active effects.
- Keyboard focus indication.
- Disabled-state styling.
- Responsive button sizing for smaller screens.

The form is displayed below the student profile when activated. The existing dashboard styling and profile layout are preserved.

2.5 Implementation Logic

The implementation follows these steps:

- The student logs in using the existing authentication process.
- The Student Dashboard loads the student's profile.
- The Change Password form remains hidden by default.
- The student clicks the Change Password button.
- The application displays the password change form.
- The student enters the current password.
- The student enters the new password and confirmation.
- The frontend checks the submitted values.
- If client-side validation fails, an error message is displayed and the request is not submitted.
- If client-side validation succeeds, the frontend sends a PATCH request to the backend.
- The backend verifies the authentication token and STUDENT role.
- The backend validates the submitted password information.
- The backend retrieves the stored password hash for the authenticated student.
- The backend verifies the current password.
- If the current password is incorrect, the password is not changed.
- If validation succeeds, the backend hashes the new password.
- The backend updates the existing password_hash field.
- The frontend displays a success message and clears the password fields.
- The student can close the form using Cancel or the toggle button.
- Existing student profile and logout functionality remain available.

2.6 Error and Data Handling

The implementation follows these data-handling rules:

- **Missing fields:** Reject the request and return an appropriate validation message.
- **Weak new password:** Reject the request without changing the stored password.
- **Password confirmation mismatch:** Reject the request.
- **Incorrect current password:** Reject the request without updating the password hash.
- **New password identical to current password:** Reject the request.
- **Unauthenticated request:** Reject access through the existing authentication middleware.
- **Non-student account:** Deny access to the student-specific password change route.
- **Successful password change:** Store the hash of the new password.
- **Form cancellation:** Close the form without sending a password change request.
- **Successful response:** Display a success message and clear the password fields.
- **Existing student information:** Preserve profile data and existing records.
- **Existing authentication:** Retain the current login and token-generation mechanisms.

The password change request uses the authenticated student's identity and does not permit a student to specify another account's user ID.

3. Testing and Verification

3.1 Testing Approach

The implementation should be tested through the frontend, backend API, authentication middleware, and database.

Testing should confirm that the Change Password button controls form visibility, valid password changes are processed correctly, invalid requests are rejected, and existing application functionality remains operational.

Testing Types

- Functional testing
- UI testing
- Client-side validation testing
- Backend validation testing
- Authentication and authorization testing
- API testing
- Database testing
- Error-handling testing
- Regression testing

Environment: Development

Browser: Google Chrome / Microsoft Edge

Frontend: React and Vite

Backend: Node.js and Express.js

Database: MySQL

Authentication: JWT

Password Hashing: bcryptjs

Test Roles: Student and Administrator

3.2 Test Cases

The following table defines the required tests. The actual results and status should be updated after each test has been executed.

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open the Student Dashboard.	The profile is displayed and the password change form is hidden.	To be recorded after testing.	Met
TC-02	Click the Change Password button.	The password change form becomes visible.	To be recorded after testing.	Met
TC-03	Click the toggle button again.	The password change form closes.	To be recorded after testing.	Met
TC-04	Click Cancel in the form.	The form closes without submitting a password change request.	To be recorded after testing.	Met
TC-05	Submit the form with empty fields.	The request is rejected with required-field validation messages.	To be recorded after testing.	Met
TC-06	Enter a new password shorter than eight characters.	The weak password is rejected.	To be recorded after testing.	Met
TC-07	Enter a password without an uppercase letter.	The password is rejected by the configured strength validation.	To be recorded after testing.	Met
TC-08	Enter a password without a lowercase letter, number, or special character.	The password is rejected when a required character type is missing.	To be recorded after testing.	Met
TC-09	Enter different values in the new password and confirmation fields.	The request is rejected because the values do not match.	To be recorded after testing.	Met
TC-10	Enter an incorrect current password.	The password remains unchanged and an error message is displayed.	To be recorded after testing.	Met

TC-11	Enter a new password identical to the current password.	The request is rejected.	To be recorded after testing.	Met
TC-12	Change the password using valid credentials and a strong new password.	The password hash is updated and a success message is displayed.	To be recorded after testing.	Met
TC-13	Log in using the new password after a successful change.	Authentication succeeds using the new password.	To be recorded after testing.	Met
TC-14	Attempt to log in using the old password after the change.	Authentication fails for the old password, provided no other credential change has occurred.	To be recorded after testing.	Met
TC-15	Send the password change request without a valid token.	The backend rejects the unauthenticated request.	To be recorded after testing.	Met
TC-16	Attempt to access the student password route as an administrator.	The student-only route denies access to the administrator role.	To be recorded after testing.	Met
TC-17	Verify the stored password after a successful change.	The database contains a password hash, not the plaintext password.	To be recorded after testing.	Met
TC-18	Navigate through the Student Dashboard after the change.	Profile display and Logout functionality continue to work.	To be recorded after testing.	Met

3.3 Test Summary

Test Result	Count
Total Test Cases	18
Passed	To be recorded
Failed	To be recorded
Pending Verification	18 before execution
Pass Rate	To be calculated after testing

Overall Test Result: Pending final execution and verification.

The implementation can be considered fully verified after the relevant test cases have been executed and their actual results documented.

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	The password change form is hidden when the dashboard initially loads.	UI inspection	Met
AC-002	Clicking Change Password opens the form.	UI functional testing	Met
AC-003	Clicking the toggle button again closes the form.	UI functional testing	Met
AC-004	The form provides current password, new password, and confirmation fields.	UI inspection	Met
AC-005	Required-field validation is enforced.	Frontend and API testing	Met
AC-006	The configured password strength requirements are enforced.	Validation testing	Met
AC-007	The confirmation password must match the new password.	Validation testing	Met
AC-008	The current password must be verified before changing the password.	API and authentication testing	Met
AC-009	The new password is securely hashed before being stored.	Code and database verification	Met
AC-010	Only the authenticated student can change	Authorization and API testing	Met

	their own password through this route.		
AC-011	The form displays suitable success and error messages.	UI and API testing	Met
AC-012	Existing login and student profile functionality remain operational.	Regression testing	Met
AC-013	The existing database schema is reused without requiring a new table or column.	Database schema inspection	Met
AC-014	The password change form can be cancelled without changing the password.	UI and API testing	Met

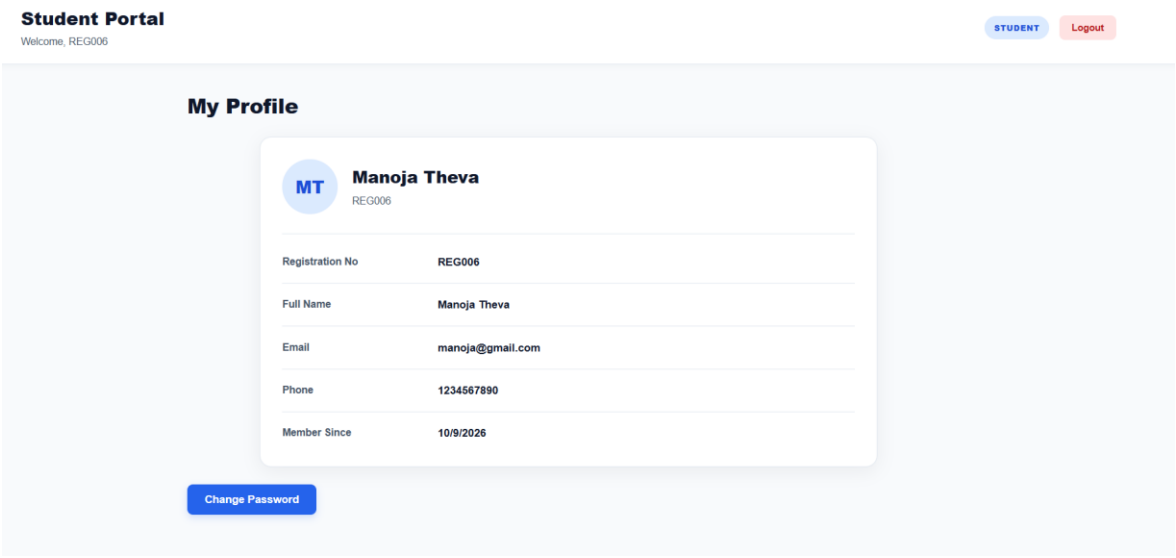
5. Evidence

Evidence 01 – Student Dashboard Before Opening the Form

Related Acceptance Criteria: AC-001

Description: Change Password button while the password change form is hidden.

Figure 1: Student Dashboard with the Change Password button.



Evidence 02 – Password Change Form Display

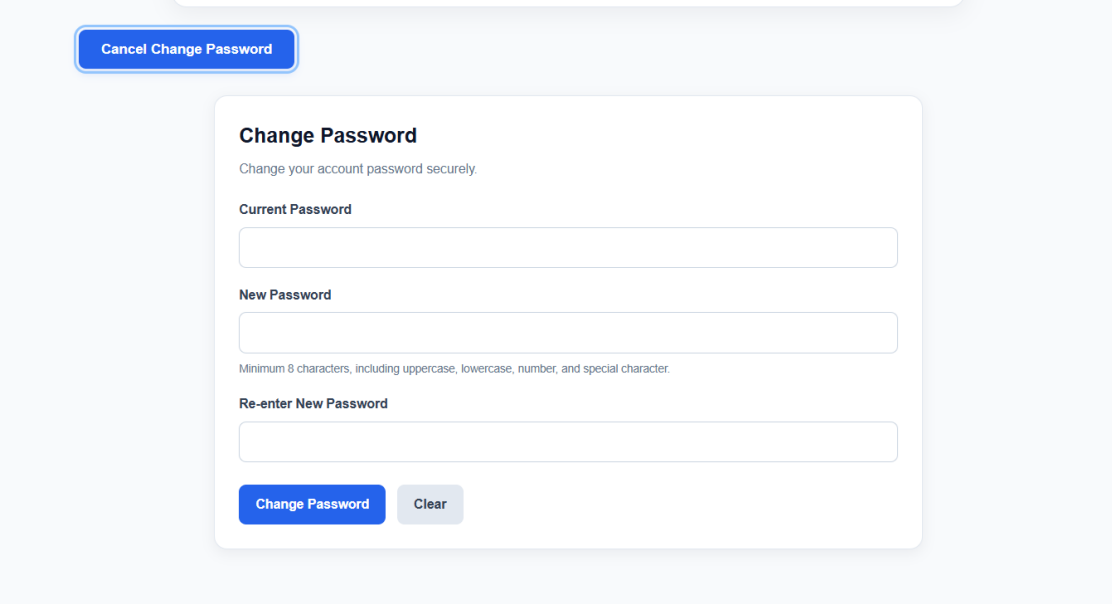
Related Acceptance Criteria: AC-002, AC-004

Description: Insert a screenshot showing the password change form after the student clicks the Change Password button.

The screenshot should show:

- Current Password field.
- New Password field.
- Confirm New Password field.
- Change Password submission button.
- Cancel button.

Figure 2: Password change form displayed after clicking the button.

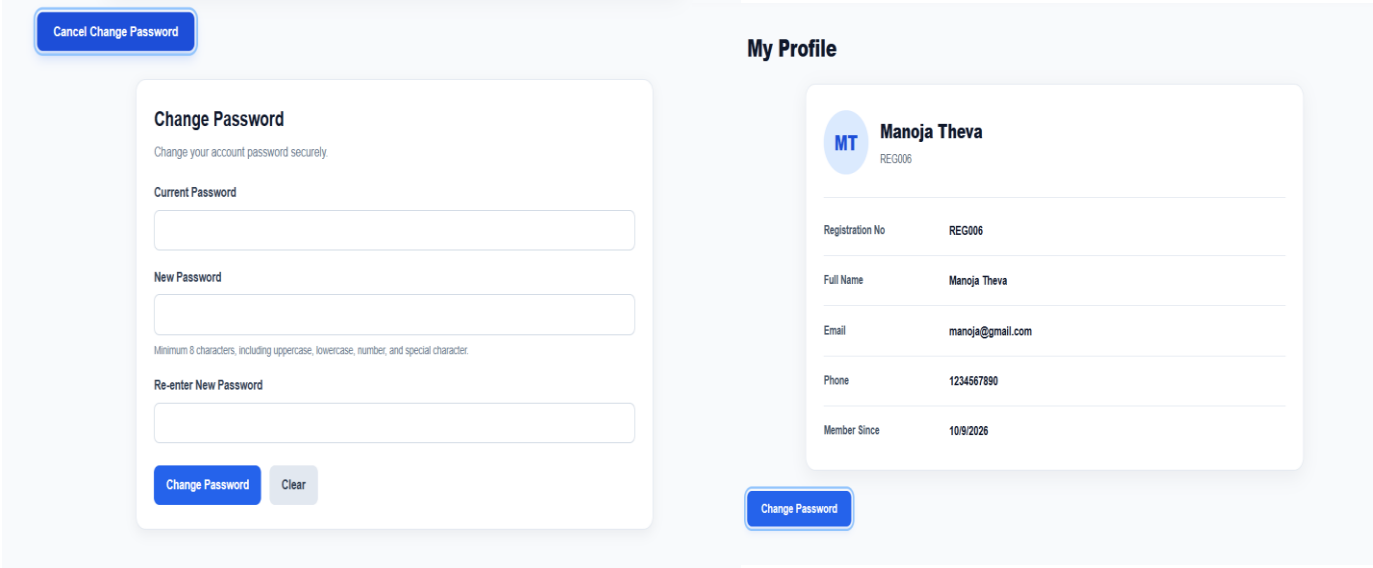


Evidence 03 – Password Form Toggle and Cancellation

Related Acceptance Criteria: AC-003, AC-014

Description: Insert screenshots demonstrating that the form closes when the student clicks Cancel or the toggle button.

Figure 3: Password change form closed after cancellation.

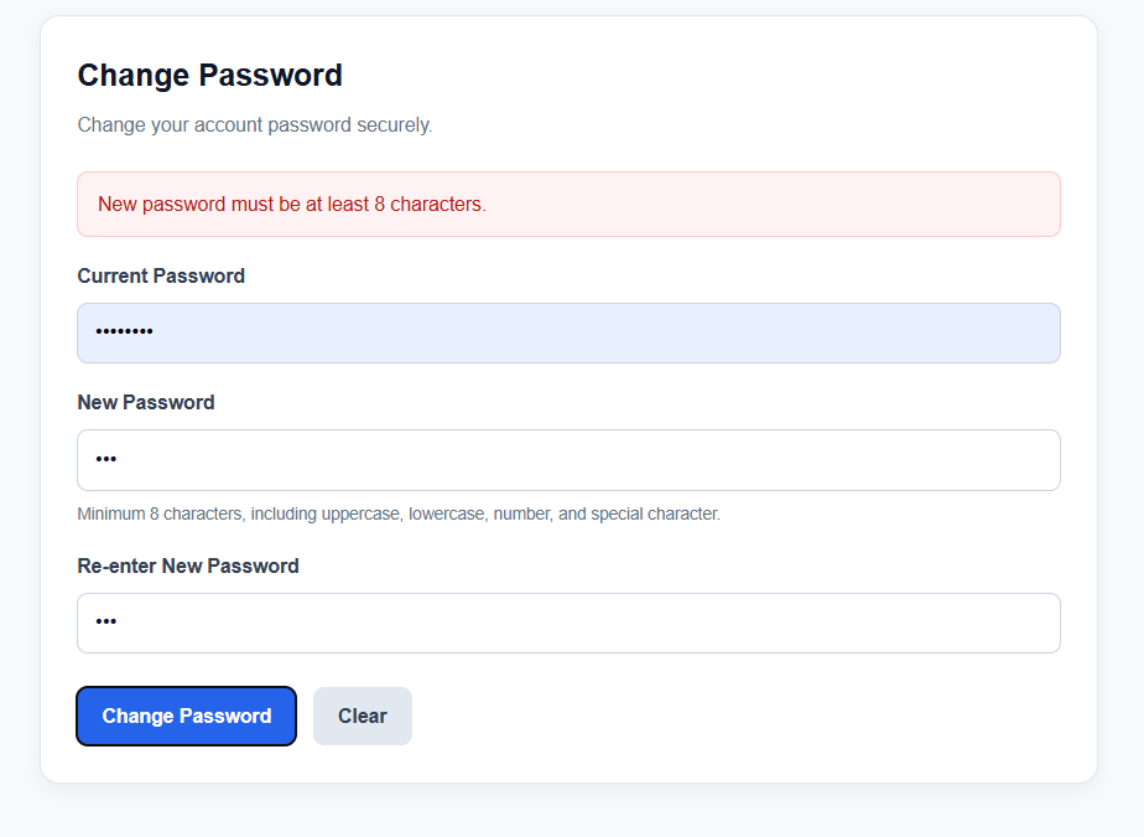


Evidence 04 – Password Validation

Related Acceptance Criteria: AC-005, AC-006, AC-007

Description: Insert screenshots showing the validation messages for a weak password and mismatched confirmation password.

Figure 4: Password strength and confirmation validation.



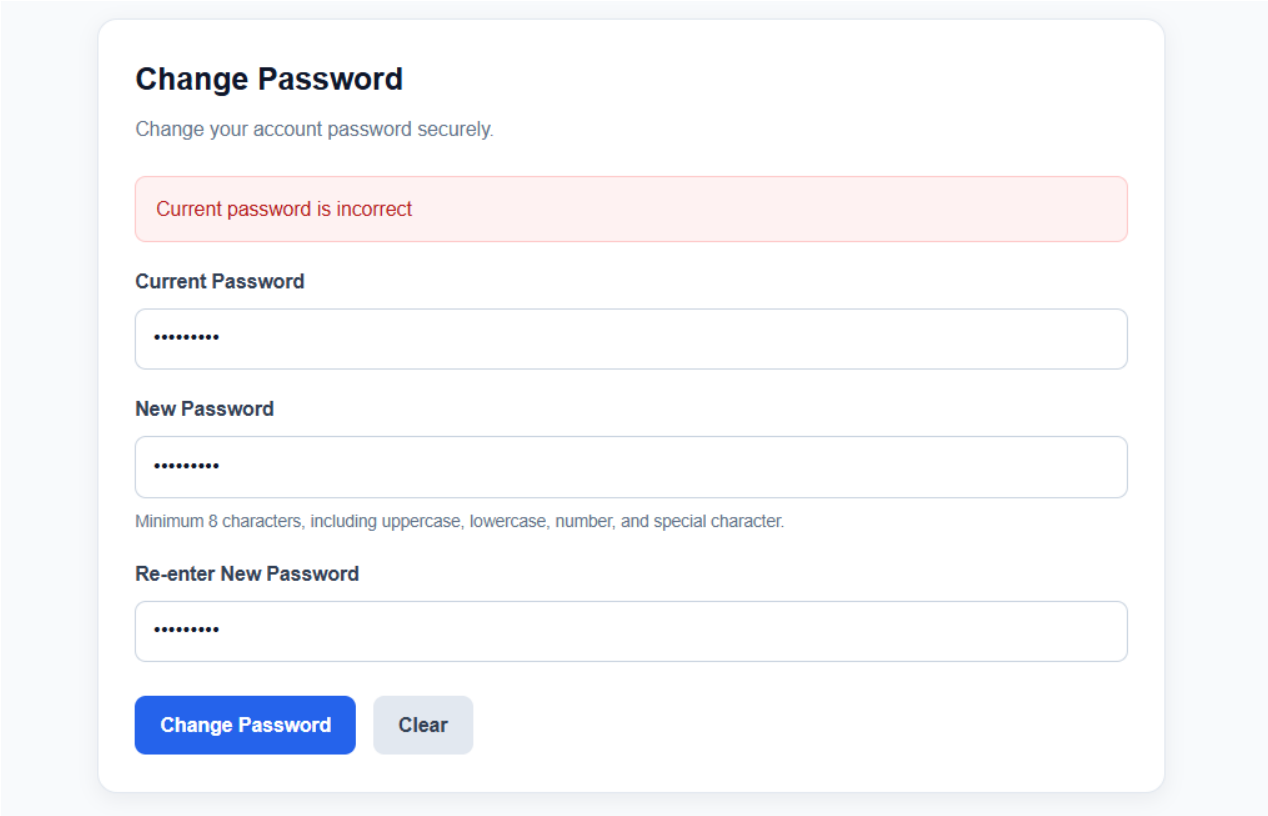
Evidence 05 – Incorrect Current Password

Related Acceptance Criteria: AC-008

Description: Insert a screenshot showing an unsuccessful password change attempt caused by an incorrect current password.

The screenshot should demonstrate that the operation is rejected and the existing password remains unchanged.

Figure 5: Password change rejected because the current password is incorrect.

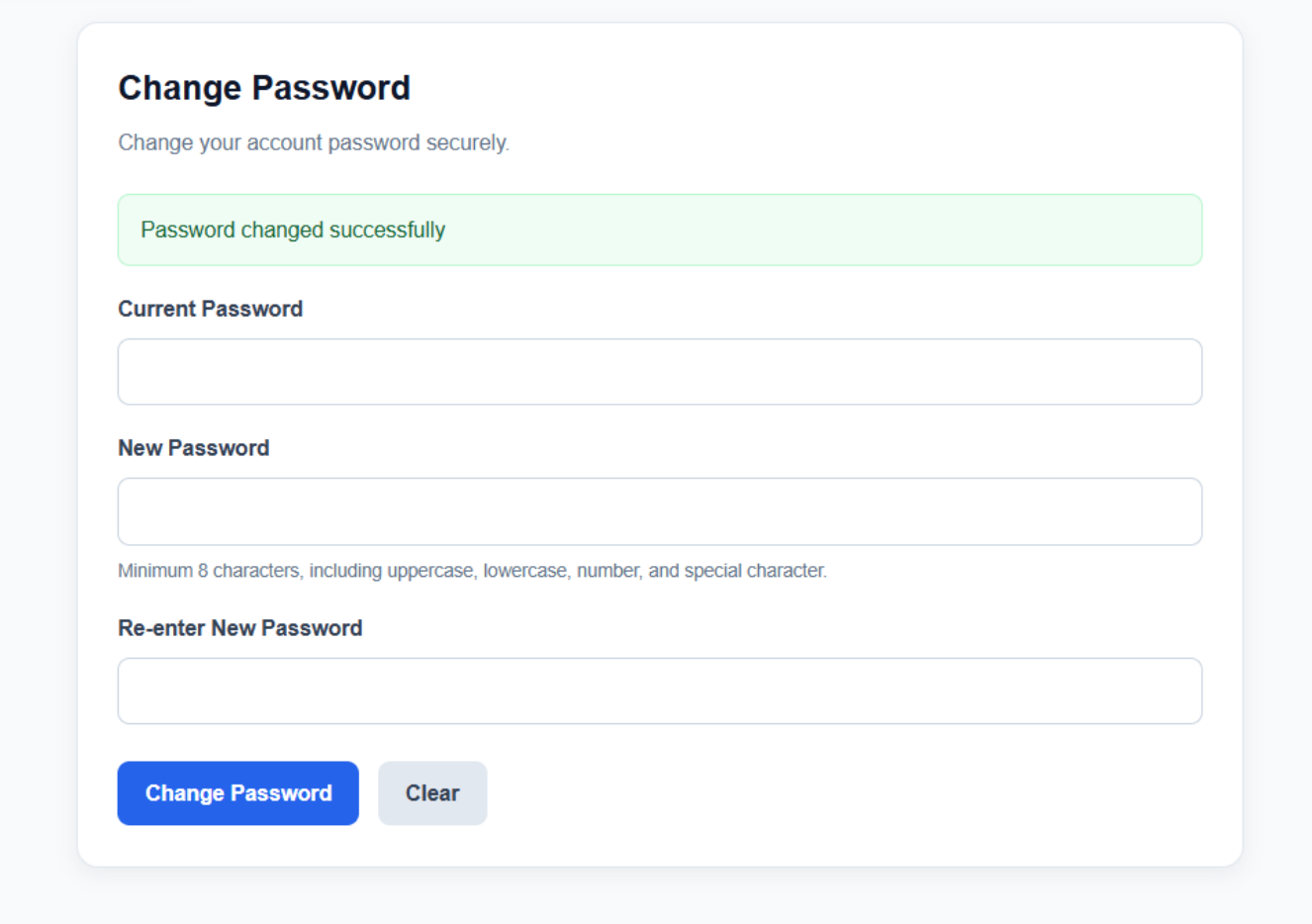


Evidence 06 – Successful Password Change

Related Acceptance Criteria: AC-009, AC-011

Description: Insert a screenshot showing the success message after the student submits valid current-password credentials and a valid new password.

Figure 6: Successful student password change.

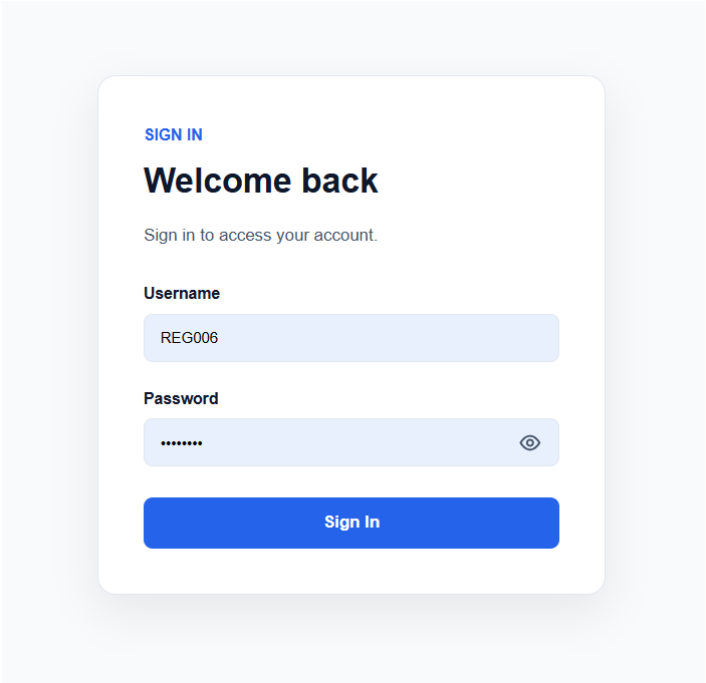


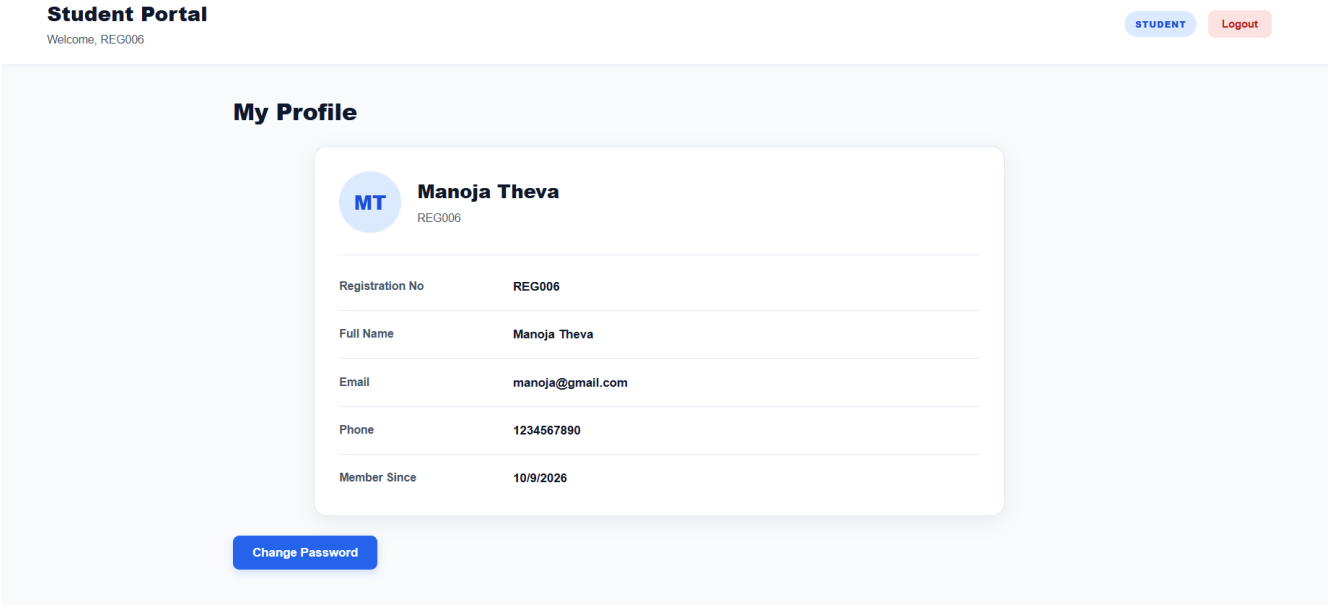
Evidence 07 – Authentication with the New Password

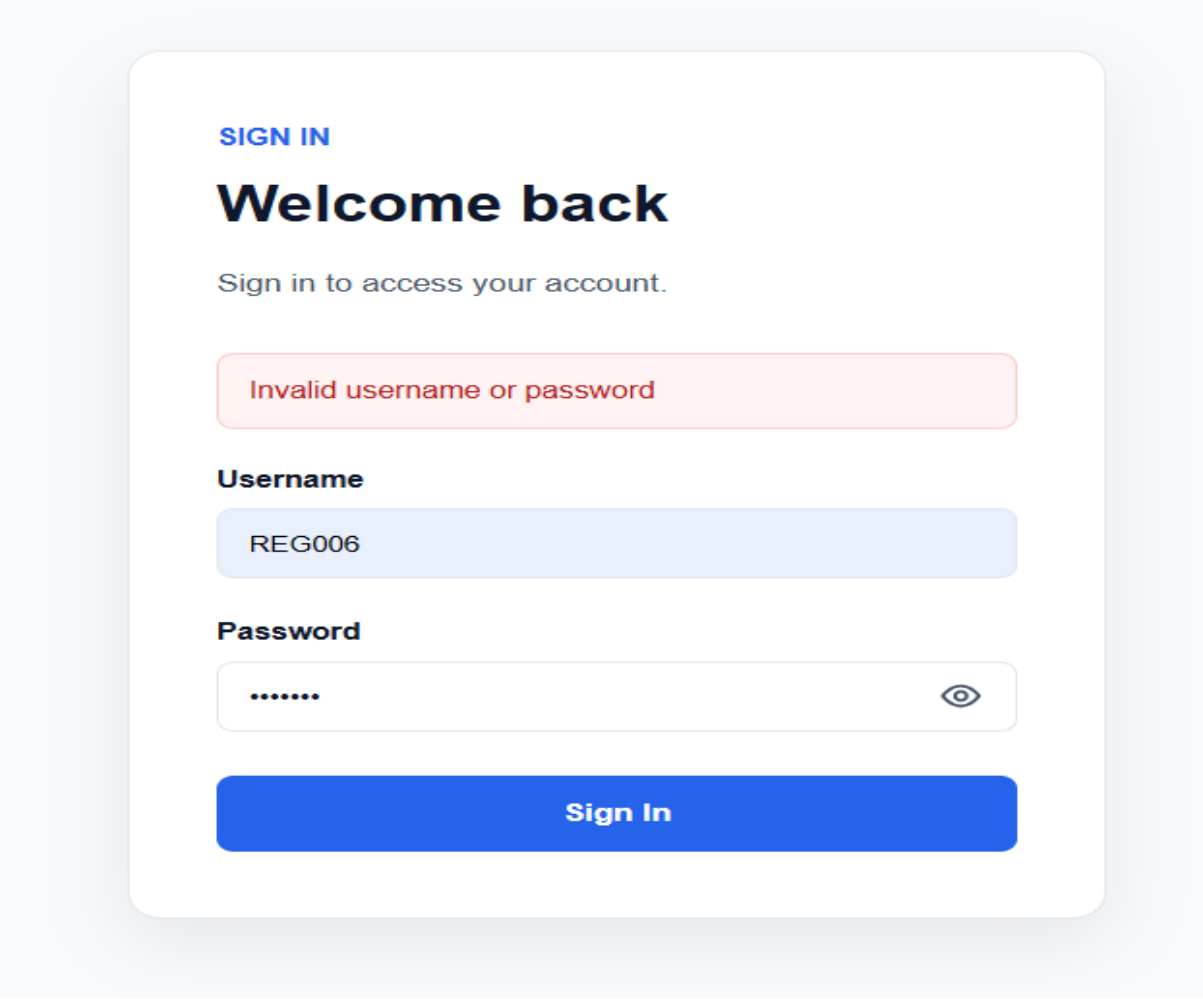
Related Acceptance Criteria: AC-008, AC-012

Description: Insert screenshots showing successful login using the new password and unsuccessful login using the old password after the change.

Figure 7: Verification of the updated student credentials.







6. Final Result

Item	Result
Requirements	Implemented in code; final verification pending
Frontend Implementation	Completed
Backend Implementation	Completed
Database Schema Changes	Not required
Password Validation	Implemented; test verification pending
Authentication and Authorization	Existing middleware reused; test verification pending
Password Hashing	Implemented using bcryptjs
Form Visibility Toggle	Implemented
Success and Error Handling	Implemented
Testing	Pending actual execution and evidence collection
Acceptance Criteria	Pending final verification
Existing Functionality	Regression verification pending
Final Status	Implementation completed; acceptance pending verification

6.1 Final Summary

CR-020 introduces a self-service password change facility for student accounts in the Student Management System.

The Student Dashboard has been enhanced with a Change Password button. The form remains hidden when the dashboard initially loads and is displayed only when the student clicks the button. The student can close the form using the Cancel button or the toggle button.

The new password change component provides fields for the current password, new password, and confirmation password. Client-side validation helps identify incomplete fields, weak passwords, and confirmation mismatches before the request is submitted.

The backend introduces the authenticated PATCH /api/students/me/password endpoint. The route restricts access to authenticated students. The controller validates the submitted values, verifies the current password using bcryptjs, hashes the new password, and updates the existing users.password_hash field.

The implementation reuses the existing database schema and authentication middleware. No additional database table or column is required. Existing student profile functionality, login processing, and administrator features are retained in the code.

The implementation work is completed. Final acceptance should be recorded after testing the form visibility, password validation, incorrect current-password handling, successful password changes, login using the new password, role-based access control, and existing functionality. Screenshots and test results should be added as evidence before submission.

6.2 Issues / Limitations

- The feature depends on the existing authentication middleware, student-role authorization, API service, and user model being configured correctly.
- The current password must be provided to authorize a password change.
- Client-side validation alone is not sufficient; the backend must continue to enforce password validation.
- The existing database must contain the users.password_hash field.
- The feature updates the current password but does not maintain a historical password-change log.
- The feature does not introduce a password-reset workflow for students who have forgotten their current passwords.
- Existing JWT tokens and active sessions are not automatically revoked by this implementation. If immediate session invalidation is required after a password change, a separate session-revocation mechanism should be implemented.
- Test results and the final acceptance status must reflect actual execution in the development environment.

Student Name: Y.Ugatharsini

Student ID: JF/ICT/24/33

Date: 13 October 2026

Signature: Y.Ugatharsini

~ END OF REPORT ~